

Inverse Problems in Geophysics

1D TEM inversion

2. MGPY+MGIN

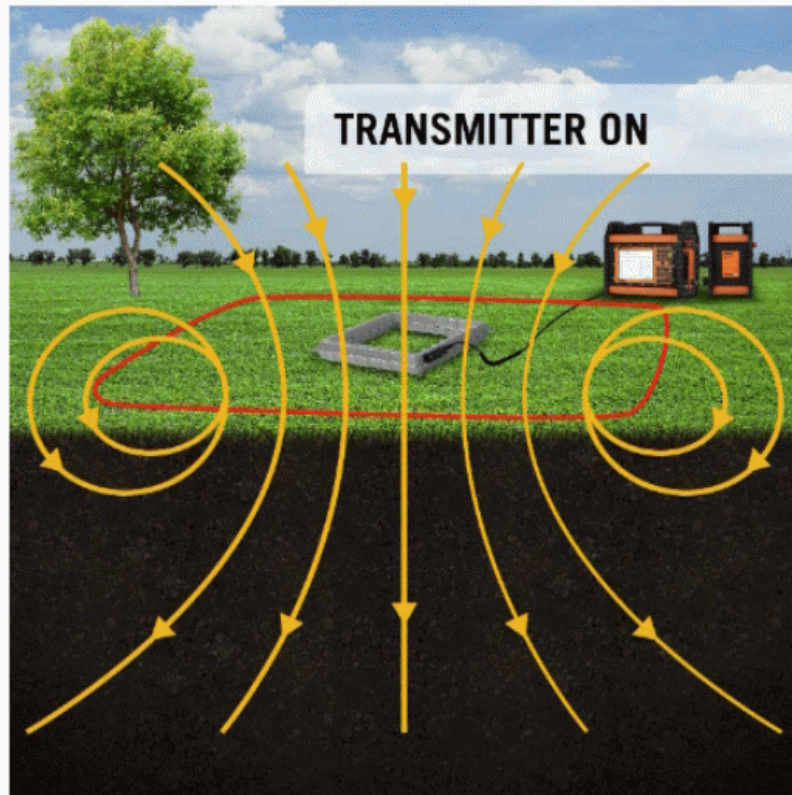
Thomas Günther

thomas.guenther@geophysik.tu-freiberg.de

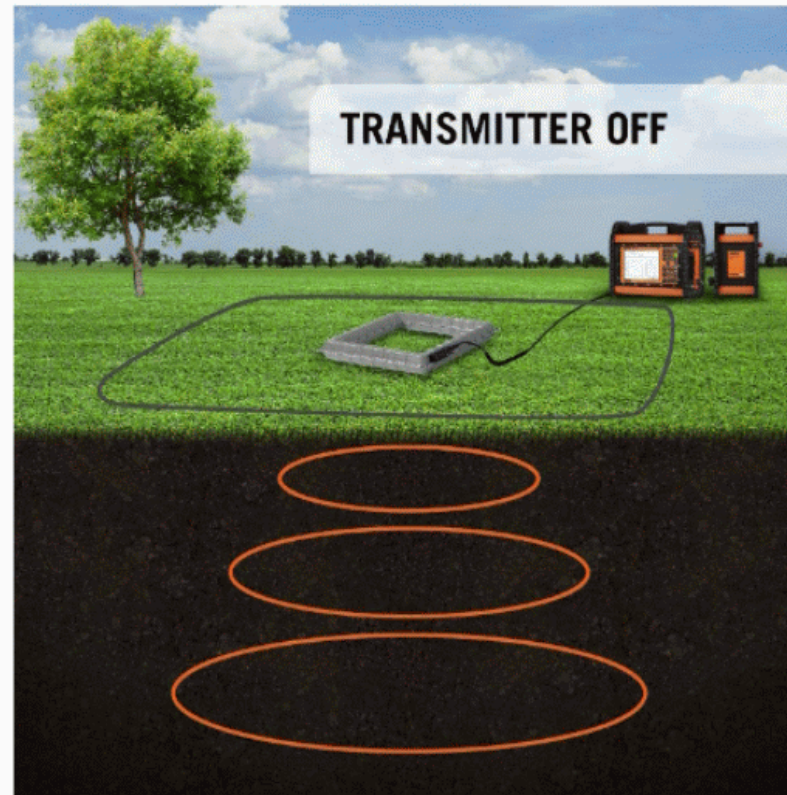
Overview

- Transient Electromagnetics (TEM)
 - playing around, apparent resistivity
- Smoothness (many layers) vs. Marquardt (few layers) approaches
- logarithmic transformations of model and data
-

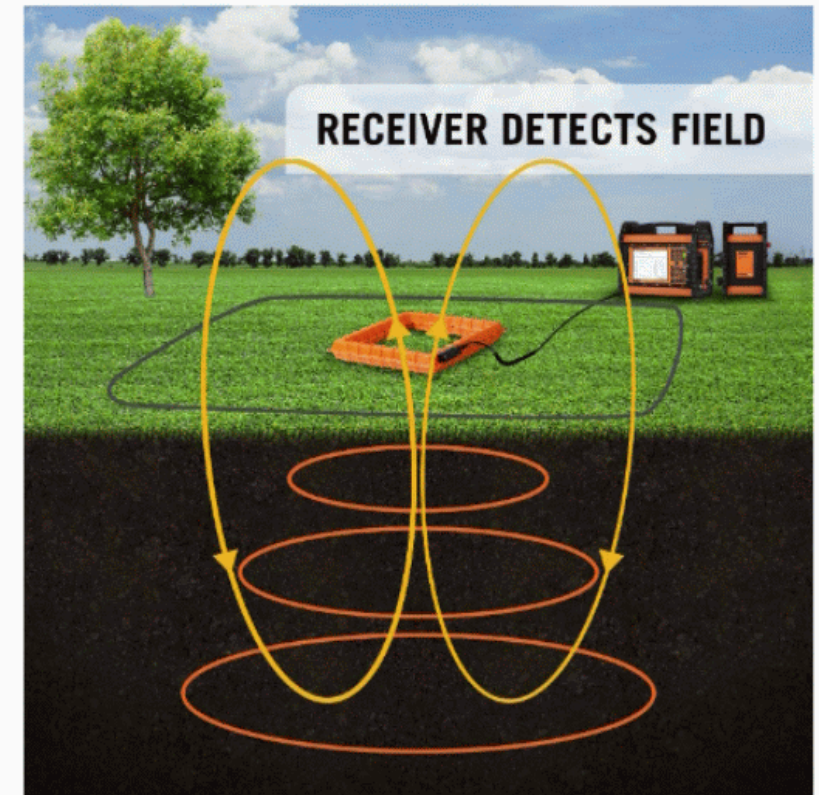
TEM measuring scheme



Current flowing through TX Loop (red) create a magnetic field (yellow)



The collapsing magnetic field creates electrical currents in the ground (orange)



Ground currents create a secondary magnetic field (yellow) recorded by the RX coil (centre)

TEM modelling with empymod

```
1 from empymod import bipole
2 time = np.logspace(-6, -2, 41)
3 L = 40
4 tx = [-L/2, L/2, L/2, -L/2, -L/2]
5 ty = [-L/2, -L/2, L/2, L/2, -L/2]
6 out = bipole(res=[2e14, 100, 100],
7             depth=[0, 30],
8             src=[tx[:-1], tx[1:], ty[:-1], ty[1:], 0, 0],
9             rec=[0, 0, 0, 0, 90],
10            signal=-1,
11            mrec="b",
12            freqtime=time, # Wanted times.
13            srcpts=3,
14            ftarg={"dlf": "key_81_2009"}, # Shorter, faster filters.
15            htarg={"dlf": "key_101_2009", "pts_per_dec": -1}).sum(axis=1)
16 plt.loglog(time, out)
```

Apparent resistivity

Tasks

1. Play around with resistivities & layer thicknesses
2. Generate a three-layer model with a good conductor in the middle
3. Generate synthetic data and add 3% Gaussian noise
4. Perform a grid search over thickness & resistivity of middle layer
5. Invert the synthetic data using Gauss-Newton method
 - for the few-layer problem (Levenberg-Marquardt algorithm)
 - for a (many) fixed layer problem (Smoothness constraints)

Levenberg-Marquardt algorithm

local damping of $\Delta \mathbf{m}$ (for stabilizing only)

1. Choose three-layer starting model and compute forward response
2. Compute Jacobian matrix by brute force
3. Solve for model update

$$\Delta \mathbf{m} = (\mathbf{J}^T \mathbf{J} + \lambda \mathbf{I})^{-1} \mathbf{J}^T (\mathbf{d} - \mathbf{f}(\mathbf{m}))$$

4. Update model, response, check convergence & continue with 2

Gauss-Newton with smoothness constraints

1. Define layered model and compute smoothness matrix $\mathbf{C}$
2. Choose starting model $\mathbf{m}^0$ ($n=0$) & compute model response $\mathbf{f}(\mathbf{m}^n)$
3. Compute sensitivity matrix $\mathbf{S}^n$ and display it
4. Solve linearized subproblem

$$\Delta \mathbf{m} = (\mathbf{S}^T \mathbf{S} + \lambda \mathbf{C}^T \mathbf{C})^{-1} (\mathbf{S}^T (\mathbf{d} - \mathbf{f}(\mathbf{m})) - \lambda \mathbf{C}^T \mathbf{C} \mathbf{m})$$

5. Optimize step length τ^n & update model: $\mathbf{m}^{n+1} = \mathbf{m}^n + \tau^n \Delta \mathbf{m}^n$
6. Compute model response & check convergence: proceed with 2.
 - maximum iteration number, decrease of Φ , reaching target $\chi^2 = 1$

Model transformation

To keep the parameters positive, we often invert for the logarithms.

If we invert for $\hat{m}$ instead of m , we use the chain rule

$$\frac{\partial f}{\partial \hat{m}} = \frac{\partial f}{\partial m} \cdot \frac{\partial m}{\partial \hat{m}} = \frac{\partial f}{\partial m} / \frac{\partial \hat{m}}{\partial m}$$

E.g. $\partial \log \rho / \partial \rho = 1/\rho$

Data transformation

Often, measured data show a wide range so that we use the logarithm

If we invert $\hat{d}$ instead of d , we use the chain rule

$$\frac{\partial \hat{f}}{\partial m} = \frac{\partial f}{\partial m} \cdot \frac{\partial \hat{f}}{\partial f}$$

E.g. $\partial \log \rho^a / \partial \rho^a = 1/\rho_a$

Combined model and data transformation

$$\text{Sensitivity } S_{ij} = \frac{\partial \rho_i^a}{\partial \rho_j}$$

$$\text{Data } d_i = \log \rho_i^a, \text{ model parameter } m_i = \log \rho_j$$

⇒ Jacobian matrix

$$J_{ij} = \frac{\partial \log \rho_i^a}{\partial \log \rho_j} = \frac{\partial \rho_i^a}{\partial \rho_j} \cdot \frac{\rho_j}{\rho_i^a}$$

Steps

1. Start with a homogeneous resistivity
2. Compute the sensitivity